Utilizarea limbajului Verilog în proiectarea hardware-ului

Semnale

- elemente de bază în orice implementare fizică
- fiecare semnal poartă 1 bit de informație
- deci în mod fundamental lucrăm cu biți
  - sau cu tablouri de biți

Tipuri de semnale (1)

- pot fi declarate în diverse moduri
  - în funcție de unghiul din care le privim
- intrare-ieșire pentru module
  - `input` - semnal de intrare pentru modulul curent
  - `output` - semnal de ieșire pentru modulul curent
  - `inout` - atât intrare, cât și ieșire

Tipuri de semnale (2)

- semnale interne modulelor
  - `reg` - poate memora valori preluate la anumite momente
    - uzual - implementare secvențială
  - `wire` - realizează legătura între intrarea unui circuit și ieșirea altuia
    - nu poate memora valori
- cele două categorii se pot suprapune

Tipuri de semnale (3)

- semnalele declarate ca `input` sunt automat de tip `wire`
  - nu putem impune valori unui semnal de intrare
- semnalele declarate ca `output` sunt de tip `wire` în mod implicit
  - pot fi conectate combinațional la alte semnale, de exemplu prin `assign`
  - le putem declara ca `reg` în mod explicit

Exemplu

```
module semnale(i,e1,e2);
input i;
output e1,e2;
reg x,e2;
assign e1=~x;
...
endmodule
```

Exemplu (cont.)

- semnalul `i` este intrare, deci nu poate fi `reg`
- semnalele `e1` și `e2` sunt ieșiri
- `e2` este `reg`
  - poate fi modificat direct
- `e1` nu este declarat `reg`
  - poate fi modificat prin intermediul semnalului `x`

Depinde de unde privim semnalul

- același semnal poate fi ieșire pentru un circuit (modul) și intrare pentru altul
  - situația apare de fapt foarte des
- deci și modul de declarare a semnalului poate diferi între cele două module
  - un modul îl poate modifica, celălalt îi poate doar citi valoarea

Exemplu

```
module iesire(s_out);
output s_out;
reg s_out;
...
s_out = 1;
...
endmodule
```

Exemplu (cont.)

```
module intrare(s_in);
input s_in;
reg x; // semnal intern
...
x = s_in;
...
endmodule
```

Exemplu (cont.)

```
module main();
wire s;
iesire a(s);
intrare b(s);
...
endmodule
```

Exemplu (cont.)

- ieșirea primului modul este conectată la intrarea celui de-al doilea
  - deci este de fapt același semnal
  - primul modul îi modifică valoarea
  - al doilea poate doar să o citească
  - la fel și modulul `main`
    - în mod obișnuit nu putem modifica valoarea unui semnal de ieșire a unui modul din afara acestuia

Semnale `inout`

- reutilizarea aceloraşi pini pentru intrări şi ieşiri
  - la momente diferite de timp
  - util doar la module top-level
- de fapt sunt componente diferite care lucrează cu intrarea, respectiv ieşirea
  - ceea ce scoatem pe ieşire nu trebuie să interfereze cu ceea ce primim pe intrare

Exemplu

```
module in_out(Clk,data);
input Clk;
inout [7:0]data;
reg [7:0]d,x;
assign data=d;
initial
  d<=8'bzzzzzzzz;
```

Exemplu (cont.)

```
always @(posedge Clk)
begin
  x=data;
  #5 d<=x+1;
  #5 d<=8'bzzzzzzzz;
end
endmodule
```

Exemplu (cont.)

- semnalul `data` privit ca ieșire
  - legat (combinațional) la ieșirea variabilei `d`
  - deci orice modificare a valorii `d` se regăsește automat pe ieșirea `data`
  - când nu avem nimic de transmis pe ieșire, `d` trebuie să fie în înaltă impedanță
- semnalul `data` privit ca intrare
  - este citit în variabila `x` la anumite momente

Exemplu (cont.)

- atunci când scriem la ieșire, semnalul de la intrarea `data` trebuie să fie la rândul său în înaltă impedanță
  - provine de la alt modul
  - deci trebuie să existe un protocol de comunicare
  - sarcina proiectantului

Operații cu semnalele

- operatorii - sintaxă similară cu limbajul C
- operațiile pe biți - mult mai des utilizate
- unele operații pe biți se pot realiza mult mai eficient în hardware
- dacă un singur bit dintr-o variabilă are valoarea `x` (nedeterminat), toată variabila are valoarea `x`

Operatori de reducere

- funcţii booleene de aritate 2 (sau mai mare)
- operatori: `| & ^`
- pot fi aplicaţi asupra biţilor dintr-un vector
- exemplu: test dacă o variabilă are valoarea 0
  - `reg [7:0]a;`
  - `reg b;`
  - `b <= ~(|a);`

Tipuri auxiliare

- `integer`
- `real`
- `time`
- variabilele din aceste tipuri folosite nu sunt pentru sinteza circuitelor, ci pentru
  - exprimarea operațiilor paralele
  - simulare

Șiruri de caractere

- nu există un tip dedicat
- constante - între caracterele "..."
- variabile - declarate ca `reg`
  - trebuie să conțină un număr suficient de biți
  - dacă sunt mai mulți - completate cu 0 la dreapta

```
reg [20*8:1]s; // 20 de octeti
s="sir de caractere";
```

Bucle

- implementare
  - nivel jos
  - test + salt
- implementarea salturilor
  - prin stabilirea stării următoare
- structuri repetitive (`while`/`for`/`repeat`)
  - nu sunt destinate descrierii buclelor secvenţiale
  - rol - descrierea operaţiilor care se pot realiza în paralel

Exemplu

- inversarea ordinii biților într-o variabilă

```
reg [7:0]a;
integer i;
a<=8'b01010000;
#5 for(i=0;i<8;i=i+1)
  a[i]<=a[7-i];
```

Exemplu (cont.)

- atenție la precizarea întârzierilor

```
a<=8'b01010101;
for(i=0;i<8;i=i+1)
   #5 a[i]<=a[7-i];
```

- în acest caz, fiecare atribuire din buclă se face la 5 unități de timp după cea anterioară
- nu se fac în paralel - rezultat incorect